

AUDIT BOT

Current-State Product Requirements Document

How the app works, how Shopify data is fetched, and how to improve speed and reliability

Document purpose

This document explains the live implementation in simple language. It separates what already exists from recommended future improvements, so product and engineering decisions can be made without confusing proposals with current behaviour.

Prepared from the codebase on 14 September 2026

Status: Current-state PRD + optimization decision guide

Document control

Item	Value
Product	Audit Bot — embedded Shopify analytics application
Primary users	Shopify merchants, DTC founders, e-commerce operators and analysts
Current reporting pages	Product Audit and New Arrival Analysis
Data sources	Shopify Admin GraphQL, ShopifyQL analytics, local Prisma/SQLite cache
Reporting availability	Last 18 full calendar months through the allowed current date
Document scope	Current behaviour, data lineage, calculations, risks and recommended performance roadmap

Contents

- 1. Product vision and users
- 2. Current application scope

- 3. System architecture
- 4. Shared data foundations
- 5. Product Audit workflow
- 6. New Arrival Analysis workflow
- 7. Metric and calculation rules
- 8. Current loading and error controls
- 9. Root-cause analysis
- 10. Optimization options and consequences
- 11. Recommended implementation roadmap
- 12. Acceptance criteria and operating checklist

1. Product vision and users

Audit Bot helps merchants inspect product-level inventory, sales, landing sessions and new-arrival cohort performance without manually exporting Shopify reports into spreadsheets.

Primary jobs to be done

- Understand how each product performed during a chosen date range.
- Compare sales, orders, traffic and inventory at product level.
- Identify when products entered a new-arrival cohort and measure their performance over later periods.
- Break cohort performance down by Product Type or Product Tag.
- Export report-ready data in CSV, JSONL or XML formats.

Product principles

- Use the store's default currency for every money value.
- Show a clear loading state instead of leaving stale data visible.
- Warn when Shopify returns incomplete data rather than silently presenting it as complete.
- Keep calculations understandable and document the business meaning of each metric.

2. Current application scope

Area	What exists now	What it does not do
Public landing page	Introduces the app and routes merchants into Shopify authentication.	It does not fetch analytics report data.
In-app Home	Provides navigation to Product Audit and New Arrival Analysis.	The former Order Details report has been removed.

Area	What exists now	What it does not do
Product Audit	Custom date report for product catalog, inventory, landing sessions, orders and sales metrics; configurable dimensions, filters, sorting and export.	Product page views and product sessions were intentionally removed because they were not reliable enough.
New Arrival Analysis	Monthly or weekly cohort matrix plus product-level cohort details; classification by Product Type or Product Tag.	Category tables are not prepared automatically; they load when opened.
Additional page	A basic authenticated placeholder route remains.	No major reporting workflow is currently defined there.

3. System architecture

1. Merchant opens the embedded app inside Shopify Admin

↓

2. Shopify authenticates the store and supplies an Admin API session

↓

3. React Router loader runs on the app server

↓

4. Server reads reusable catalog/analytics cache and requests missing Shopify data

↓

5. Server matches product IDs and handles, then calculates report rows



6. Browser receives report data and renders filters, totals, table and export controls

Technology in plain language

Component	Simple explanation
React Router 7	Runs page loaders on the server and renders the embedded user interface.
Shopify Admin GraphQL	Fetches product master data such as ID, title, type, tags, handle, status, image and created date.
ShopifyQL	Fetches analytics tables for inventory, sales and sessions.
Prisma + SQLite	Stores Shopify sessions and reusable report/cache records.
App Bridge / Shopify components	Keeps the app embedded and visually aligned with Shopify Admin.

4. Shared data foundations

4.1 Product catalog

The catalog supplies descriptive fields that analytics rows often do not contain reliably: Product ID, title, type, tags, handle, status, image and created date.

1. Check ProductCatalogCache for this shop



2. If cache exists, return it immediately

↓

3. If older than 6 hours, refresh in the background while serving stale cache

↓

4. Refresh with Shopify Bulk Operation; fall back to pages of 250 products if bulk fails

↓

5. Store refreshed catalog in SQLite for later requests

Why the refresh time can look old

A cache older than six hours is intentionally served immediately so the report does not wait for the entire catalog download. A background refresh begins, so the displayed timestamp can remain old until that refresh completes and the page is opened again.

4.2 Analytics cache

Each analytics cache key contains the shop, dataset name, start date and end date. Only successful, non-truncated results are saved.

- Historical ranges older than the three-day finalization grace period can be cached.
- Recent ranges are fetched live because Shopify can still update them.
- A cache hit avoids the ShopifyQL request; a miss runs the query and saves it if complete.
- Different query shapes use different dataset keys. Product Audit daily inventory cannot automatically replace New Arrival monthly inventory without a deliberate shared data model.

4.3 Error and completeness handling

- ShopifyQL queries have a 100,000-row limit.
- Temporary errors such as throttling, timeout, 502, 503 and 504 are retried up to three times with increasing pauses.
- Errors do not crash the full report. The page can show available sections plus an incomplete-data warning.
- Debug details show request IDs, rows, time, attempts, cache status and truncation information.

5. Product Audit workflow

5.1 User experience

- Choose a custom range within the allowed 18-month window.
- Apply the range; the selected dates appear immediately and the old report body is replaced by a loader.
- Choose dimensions such as Product ID, Title, Status, Type, Tags, Month, Week or Day.
- Choose metrics, reorder them, filter values and sort columns.
- View 50 rows per browser page or export the current page/all filtered results.

5.2 Queries currently run

Dataset	ShopifyQL shape	Purpose
Landing sessions	Sessions grouped by day and landing page path where landing page type is Product.	Maps a product landing URL/handle to landing sessions and checkout-session metrics.
Sales breakdown	Sales grouped by day and product ID.	Orders, quantities, reversals, gross/net/total sales, discounts, shipping, fees and taxes.
Unique order total	Sales orders without product grouping.	Selected-metric total counts a store order once even when it contains multiple products.
Inventory	Starting inventory, ending inventory and first day in inventory grouped by day and product ID.	Supports range boundaries and optional Month/Week/Day grouping.

5.3 Inventory chunking

Inventory is the largest dataset. The selected range is divided into non-overlapping seven-day pieces. Up to two pieces start together. If a piece reaches 100,000 rows, it is divided again until complete or until one day remains.

1. Split selected range into seven-day chunks



2. Read cached historical chunks; fetch missing/live chunks



3. If a chunk reaches 100,000 rows, divide it into smaller chunks



4. Pause between live requests to reduce throttling



5. After normal retries, wait and replay only rate-limited chunks sequentially



6. Combine all successful daily rows

Important limitation

Chunking prevents a long range from hitting 100,000 rows as one giant response. It does not remove Shopify's request-rate limit. A very long uncached range can still require many requests and therefore needs pacing, recovery and eventually a stronger storage design.

5.4 Server matching and browser processing

- Product IDs connect sales and inventory rows to the catalog.
- Landing-page handles connect session paths to catalog products.
- The server creates product-day source rows and sends them to the browser.
- The browser currently groups those rows by selected Month, Week, Day and product dimensions using a streaming single-pass calculation.
- Changing a dimension shows a loader before regrouping to avoid an apparently frozen screen.

6. New Arrival Analysis workflow

6.1 User experience

- Choose Month or Week grouping. Month is the default.
- Choose Product Type or Product Tag classification. Product Type is the default.
- View the Overall cohort matrix first.
- Category names are listed in collapsed form. A category is calculated only when clicked, then retained in browser memory for instant reopening.
- Open Cohort Details for product-level values, search, filters, sorting, density and export.

6.2 Monthly fetch

For each selected month, the server runs four period-level ShopifyQL queries in parallel: product inventory, product sales/orders, total store sales and product landing sessions. Up to two months are processed concurrently.

6.3 Weekly fetch

Selected weeks are grouped into batches of up to seven weeks. Each batch runs four queries grouped by week. The result is then separated into individual week rows in the app. If a batch reaches the row limit, it is divided into smaller batches.

6.4 First-cohort lookback

The report checks from the first day of the month two months before the selected start through the day immediately before the selected start. A product found there is moved to the first displayed cohort only if it also becomes active somewhere inside the selected report range.

Example: range starts 15 August 2025. The lookback is 1 June–14 August 2025. If Product A was active in July and becomes active again in January inside the selected range, it belongs to the August first cohort. If it never becomes active inside the range, it is excluded.

6.5 Cohort calculation

1. Combine catalog fields with inventory, sales, orders and landing sessions for every period

↓

2. Mark a product active when starting inventory > 0, ending inventory > 0, or total sales > 0

↓

3. Assign the earliest active selected period as launch cohort



4. Override to first cohort only when lookback and in-range activity rules both match



5. Calculate Overall matrix and Product Type/Tag category list



6. Calculate category matrix on click; keep loaded result in browser memory

Tag duplication rule

A product counts once in Overall. If it has multiple tags, it appears in every matching tag category. Therefore, adding category totals together can double-count the same product and should not be used as an Overall total.

7. Metric and calculation rules

7.1 Product Audit key metrics

Metric	Current meaning
Starting inventory	Inventory value from the earliest available daily row in the selected group.
Ending inventory	Inventory value from the latest available daily row in the selected group.
First day in inventory	Shopify's first_day_in_inventory value returned within the selected ShopifyQL result; earliest value is retained when grouped.

Metric	Current meaning
Landing sessions	Sessions whose journey began on that product's landing-page path.
Orders by product	Orders containing that product. One multi-product order contributes one order to each included product.
Unique orders total	Store-level order count without product grouping; the same multi-product order counts once.
Net items sold	Shopify net item quantity after reversals.
Total sales	Shopify total_sales in the store's default currency.

7.2 New Arrival matrix formulas

Metric	Simple definition / formula
NA SKUs	Active distinct cohort products in that period.
NA SKU %	$\text{Active cohort products} \div \text{total products launched in that cohort} \times 100.$
NA SKU % (Total)	$\text{Active cohort products} \div \text{all active store products in that period} \times 100.$
NA Inventory	Ending inventory in launch period; starting inventory in later periods.
NA Inventory %	$\text{Cohort inventory} \div \text{total store inventory in that period} \times 100.$

Metric	Simple definition / formula
NA Sales	Sum of total_sales generated by cohort products.
NA Sales %	Cohort total_sales ÷ store total_sales in that period × 100.
Landing Sessions	Sessions that began on a PDP belonging to the cohort.
Orders	Product-level orders summed for the cohort.
CR %	Orders ÷ landing sessions × 100. Directional only, because sessions that started elsewhere are not included.

Why first-cohort NA SKU % can be below 100%

Lookback products can be assigned to the first cohort even if they are inactive during the first displayed period. They must become active later inside the selected range, but the numerator for the first period counts only products active in that period.

8. Current loading and error controls

Control	Already implemented	Benefit
Parallel independent queries	Catalog, shop info and analytics requests start together where safe.	Total wait approaches the slowest branch instead of the sum of every branch.
Catalog stale-while-refresh	Serve existing catalog, refresh after six hours in background.	Fast initial catalog access.

Control	Already implemented	Benefit
Historical analytics cache	Successful finalized ranges stored by shop/dataset/range.	Repeat historical ranges avoid ShopifyQL.
Automatic query retry	Up to three attempts for temporary failures with increasing waits.	Recovers brief throttles/timeouts.
Inventory date chunking	Seven-day chunks, recursive split on 100k limit.	Reduces incomplete inventory responses.
Inventory recovery passes	Paced live requests; failed rate-limited chunks retried after shared cooldowns.	Recovers longer throttle windows without repeating successful chunks.
Loading screens	Old report body hidden during date/dimension changes.	Prevents users reading stale data as new data.
Streaming browser grouping	Single pass instead of retaining and repeatedly sorting group arrays.	Reduces browser freeze risk.
Category on demand	New Arrival category matrix runs only on click and stays in memory.	Faster Overall first display.

9. Root-cause analysis: why reports can still be slow

Root cause	What happens	Visible symptom
Too many raw Product Audit rows	4,003 products × 72 days can approach 288,216 rows before sessions/sales matching.	Long server response, large JSON transfer and expensive browser work.
Inventory request multiplication	A long range becomes many seven-day requests; large chunks may split again.	Long loading time and Shopify rate limiting.
Live recent dates	Recent ranges bypass analytics cache by design.	Repeated requests for the current period still call Shopify.
Exact-range cache keys	A cached Jul 1–31 result does not automatically satisfy Jul 1–Aug 15 unless matching chunks exist.	Overlapping custom ranges can repeat work.
Concurrent report families	Landing sessions, sales, unique orders and inventory begin together.	Fast when budget is available; throttling risk when inventory also creates many calls.
Browser receives more than it displays	Only 50 rows are visible, but source rows for the full range are sent for client filtering/grouping/export.	Memory pressure and Page Unresponsive warnings.
SQLite production scaling	A local SQLite file is simple but not ideal for multiple server instances or very large analytics history.	Deployment limits, lock contention and harder horizontal scaling.

Root cause	What happens	Visible symptom
Shopify service conditions	Throttling or temporary errors can happen even with correct code.	Incomplete-data warning after retries.

10. Optimization options and consequences

Option A — Server-side aggregation (highest immediate UI impact)

Today Product Audit sends daily source rows to the browser so the browser can build Month/Week/Day groups. Server-side aggregation means the server performs that grouping first and sends only the final grouped rows required for the chosen dimensions.

Example: instead of sending about 288,000 product-day rows for 4,003 products over 72 days, a product-only view can send about 4,003 rows. A product-by-month view may send about 12,000 rows.

- Benefit: far smaller response, less memory, fewer Page Unresponsive incidents, faster table rendering.
- Consequence: changing a dimension requires a server request instead of instant local regrouping.
- Implementation: send selected dimensions/filters/sort/page in URL; group on server; return summary plus one page.
- Export: run a server-side export job using the same filters rather than relying on browser-held rows.

Option B — Server-side pagination and filtering

Return only the 50 rows currently visible, plus total row count and summary. Filters and sorting run before pagination on the server.

- Benefit: browser workload becomes almost constant even for very large stores.
- Consequence: every filter, sort or page change makes a small server request.
- Requirement: summary totals must be calculated separately so they represent all filtered rows, not only the current page.

Option C — Normalize daily analytics into a database

Store one cleaned row per shop, product and day. Shopify is queried once for missing/fresh dates; all reports read and aggregate the local database.

- Benefit: custom ranges, Month/Week/Day views and New Arrival reports reuse the same daily foundation.
- Benefit: Shopify timeouts and rate limits stop being part of normal page opening.
- Consequence: database storage, background jobs, monitoring, data correction and operating cost increase.
- Recommended database: managed PostgreSQL rather than SQLite for a production multi-store SaaS.

This is different from Shopify Bulk Analytics access

The app can build its own daily store using the ShopifyQL access it already has. It does not require a special new Shopify bulk analytics API, but it does require scheduled/background processing and production database capacity.

Option D — Incremental and canonical cache chunks

Cache analytics in fixed daily or weekly blocks instead of arbitrary user-selected ranges. A custom range is assembled from reusable blocks, with only uncovered boundary dates fetched live.

- Benefit: overlapping ranges reuse much more work.
- Benefit: lower cost than a complete warehouse and compatible with the current architecture.
- Consequence: partial first/last periods require careful merging; current-period blocks need refresh/expiry rules.

Option E — Global ShopifyQL request scheduler

Use one per-shop request queue shared by all report queries. The queue controls concurrency, pauses after throttling and honors any retry-after signal when available.

- Benefit: Product Audit and New Arrival requests cannot accidentally overwhelm the same store's ShopifyQL budget.
- Consequence: an uncached report may take slightly longer in the happy path, but succeeds more reliably.
- Current recovery is local to Product Audit inventory; a global scheduler would protect every ShopifyQL dataset.

Option F — Background report jobs

For heavy custom ranges or exports, create a job, show progress, store the completed result and notify the page when ready.

- Benefit: web requests do not time out; users can leave and return.
- Consequence: requires a job queue/worker, job status storage and cleanup policy.

Option G — Precompute common summaries

Prepare common Month and Week totals after data becomes available, so normal report openings read ready-made summaries.

- Benefit: near-instant common views.
- Consequence: more stored rows and background processing; custom dimensions may still need live aggregation.

Option H — Reduce debug payload in normal use

Keep request IDs, timing and status, but do not return large raw debug structures unless debug mode is explicitly enabled.

- Benefit: smaller response and cleaner UI.
- Consequence: support staff must enable debug mode when deeper evidence is needed.

11. Optimization decision matrix

Option	Speed gain	Timeout/rate-limit gain	Complexity	Cost impact	Recommendation
Server-side aggregation	Very high	Medium	Medium	Low–medium	Do first
Server pagination/filter/sort	Very high	Low	Medium	Low–medium	Do with aggregation
Canonical cache chunks	High on repeat/overlap	High	Medium	Low–medium	Next
Global request scheduler	Neutral/slightly slower	Very high	Medium	Low	Next
Daily analytics database	Very high	Very high	High	Medium–high	Best long-term scale
Background report jobs	High perceived reliability	Very high	High	Medium	For heavy ranges/exports
Precomputed summaries	Very high for common views	High	Medium–high	Medium	After daily store

Option	Speed gain	Timeout/rate-limit gain	Complexity	Cost impact	Recommendation
Reduced debug payload	Low-medium	None	Low	Low	Quick win

12. Recommended roadmap

Phase 1 — Stabilize current live-query design

- Create a global per-shop ShopifyQL scheduler with adaptive cooldown.
- Convert cache keys to reusable canonical day/week chunks.
- Return compact debug metadata by default.
- Add query-stage timings: cache, Shopify wait, merge, serialization and browser render.

Expected result: fewer incomplete reports and clearer RCA with limited infrastructure change.

Phase 2 — Move Product Audit computation to the server

- Send selected dimensions, filters, sort and page to the loader.
- Aggregate, filter, sort and paginate on the server.
- Return only summary totals and the selected page.
- Run all-results exports on the server.

Expected result: major reduction in browser hangs and response size.

Phase 3 — Production analytics foundation

- Migrate production cache/session storage from SQLite to managed PostgreSQL.
- Store normalized shop-product-day facts with unique keys to prevent duplicates.
- Backfill historical data gradually with rate-aware workers.
- Refresh yesterday and recent mutable dates on schedule.
- Build Product Audit and New Arrival reports from the shared local fact table.

Expected result: scalable multi-store reporting, predictable page load time and minimal dependence on live ShopifyQL during page opening.

Recommended decision

Balanced path

Implement server-side aggregation/pagination and a global request scheduler first. Then introduce canonical cache chunks. Move to a managed daily analytics store when live stores or report volume justify the recurring database and worker cost.

13. Acceptance criteria

Report correctness

- Every money value uses the store's default currency.
- Product totals and unique-order totals use their documented grouping rules.
- No truncated or failed dataset is silently presented as complete.
- New Arrival first-cohort lookback and Product Tag duplication rules remain unchanged during optimization.

Performance targets for the next architecture

- Cached report first response target: under 2 seconds.
- Uncached standard report target: visible progress immediately and completion without browser unresponsiveness.
- Browser receives only the rows necessary for the current view.
- Heavy export/report work survives normal web-request timeout boundaries through background jobs.

Reliability and operations

- Per-shop queue prevents uncontrolled concurrent ShopifyQL bursts.
- Retries use bounded exponential/adaptive backoff and never loop forever.
- Successful chunks are retained when another chunk fails.
- Monitoring separates cache time, Shopify time, server calculation time and browser rendering time.

14. Plain-language glossary

Term	Meaning
Aggregation	Combining many detailed rows into totals, such as daily rows into monthly product totals.

Term	Meaning
Server-side	Work done on the app server before data is sent to the merchant's browser.
Cache	A saved successful result that can be reused instead of requesting Shopify again.
Chunk	A smaller data segment of a large request.
Rate limit / throttle	Shopify temporarily refuses requests because too many were made in a short time.
Timeout	A request took longer than the permitted waiting period.
Pagination	Returning one manageable page of rows rather than every row at once.
Background job	Long work performed separately from the page request, with progress and a stored result.
Normalized daily data	One consistent database record per shop, product and date that can be reused by multiple reports.
Directional CR	Orders divided by landing sessions; useful for comparison, but not exact product conversion because it excludes sessions that started elsewhere.

End of document